PRABODHA

MULTI-TENANT INSTITUTION MANAGEMENT PLATFORM

Technical Project Report

Architecture, API Design, Security & Development Journey of a Next.js 14 / Prisma / PostgreSQL Coaching-Institute SaaS

PREPARED BY	Ayush Kumar Prem
ROLE	Intern — Class XI Computer Science Faculty & Full-Stack Developer
ORGANIZATION	Vector Classes, Barhi, Hazaribagh
ACADEMIC AFFILIATION	B.Tech CSE + MBA (Dual Degree), DBS Global University, Dehradun
INTERNSHIP PERIOD	May 8, 2026 — Present
REPORT DATE	August 1, 2026

Table of Contents

1. Executive Summary

I have been working as a Class 11 Computer Science faculty member and full-stack developer at Vector Classes, Barhi (Hazaribagh) since May 8, 2026. Alongside classroom teaching, my core development mandate has been to architect, build, and deploy **Prabodha** — a multi-tenant SaaS platform that will eventually integrate with the institute's public-facing website.

Prabodha is a lightweight but rigorously engineered LMS/ERP for coaching institutes. It gives admins, teachers, students, and parents a single system to manage daily academic operations — attendance, timetabling, study materials, homework, and exam marks — with strict data isolation between institutes on the same deployment.

1.1 Current Project Status

The project follows a phased, agile roadmap. As of this report, **Phases 1–5 are complete** — covering the entire backend API surface, the database schema, and role-specific dashboard UIs for all four user roles. **Phases 6–7 (polish and deployment) remain open.**

Completed

- **Core architecture:** Next.js 14 App Router, Tailwind CSS design system, Prisma ORM.
- **Security & multi-tenancy:** custom JWT authentication, Role-Based Access Control (RBAC), and strict `instituteId` data isolation on every query.
- **Entity management:** full CRUD APIs and UI for Batches, Subjects, Teachers, Students, Parents, and Parent–Student linking.
- **Academic operations:** conflict-checked timetable scheduling and a fast in-class attendance workflow.
- **Content & assessments:** Study Materials, Homework (with per-student status tracking), and Exam/Marks recording.
- **Role-specific dashboards:** isolated, permission-scoped views for Admin, Teacher, Student, and Parent roles.

Remaining (Phase 6 – Phase 7)

- UX polish: skeleton loaders for data-fetching states and toast-based error handling.
- Database migration: SQLite (development) → PostgreSQL (production).
- Deployment: VPS provisioning (Ubuntu, PM2, Nginx), domain + SSL, and go-live.

1.2 Report Scope

This report documents the system architecture, database design, complete API surface, security posture, scalability strategy, and the chronological development history of Prabodha, compiled from the project's own architecture notes, API reference, roadmap tracker, and session change-log.

2. System Architecture

Prabodha uses a modern, monolithic-serverless architecture powered by **Next.js 14**. A single codebase serves both the React Server Component UI and the backend REST API via Next.js Route Handlers — eliminating the need to maintain a separate frontend and backend repository.

2.1 Technology Stack

Layer	Technology	Notes
Framework	`Next.js 14 (App Router)`	UI in `src/app/`, API routes in `src/app/api/`
Language	`TypeScript`	Strict mode enabled
Database	`SQLite (dev) / PostgreSQL (prod)`	Configured in `prisma/schema.prisma`
ORM	`Prisma`	Schema is the single source of truth for all 13 tables
Auth	`bcrypt + JWT (Bearer)`	Issued by `/api/auth/login`, verified in `lib/rbac.ts`
Styling	`Tailwind CSS`	Custom palette — Pine (green), Saffron (gold), Paper (warm white)
Validation	`Zod`	Used in every API route handler for request-body validation

2.2 Request Lifecycle

Prabodha deliberately avoids cookies — to sidestep CSRF exposure across subdomains — and instead stores the JWT client-side in `localStorage` under the key `prabodha-auth-token`. Every authenticated request follows a fixed, five-step lifecycle:

#	Stage	What happens
1	**Client action**	The apiFetch() helper attaches the JWT as an Authorization: Bearer header on every request.
2	**Middleware interception**	The API route invokes the apiHandler() wrapper, which try/catches the whole handler.
3	**Authentication**	requireAuth(request) decodes and validates the JWT signature; throws 401 if invalid.
4	**Authorization**	requireRole(user, 'admin') checks the caller's role where relevant; throws 403 if disallowed.
5	**Scoped query + response**	Every Prisma query includes where: { instituteId: user.instituteId }; results return as JSON.

2.3 Project Structure

The codebase is organised around three top-level concerns — routed pages, API handlers, and shared library code — with a dashboard route group isolating all authenticated UI:

Path	Purpose
`src/app/(dashboard)/dashboard/`	10 dashboard sections: teachers, students, batches, subjects, timetable, attendance, homework, materials, marks, settings

Path	Purpose
`src/app/api/`	7 auth endpoints plus resource routes for batches, subjects, teachers, students, assignments, timetable, attendance, materials, homework, exams, parents, and parent-student links
`src/components/dashboard/`	AdminResourcePage, DirectoryPage, ParentsPage, InstitutionStore (legacy mock, now unused by wired pages), Sidebar, StatCard, TopBar
`src/lib/`	api-client.ts, auth.ts, db.ts (Prisma singleton), rbac.ts, content-scope.ts (shared teacher scoping helpers)

Design pattern: *every API route follows the identical* `apiHandler(requireAuth → requireRole → scoped Prisma query)` *shape, keeping the 45+ endpoints structurally uniform and easy to audit.*

3. Database Design

The schema is normalised to **Third Normal Form (3NF)** and spans **13 relational models** (expanded to 15 across the project's lifetime with the later addition of Exam and Mark). Every table carries an `instituteId` foreign key, which is the project's foundational multi-tenancy guarantee: data from different coaching centres can coexist safely in one database.

3.1 Core Entities & Relationships

Entity	Role in the schema
Institute	Root tenant — has many Users, Batches, and Subjects
User	Polymorphic via a role field: admin, teacher, student, or parent
Batch	Groups students; has many BatchSubjectTeacher assignments
Subject	Taught within batches; has many BatchSubjectTeacher assignments
BatchSubjectTeacher	The central junction table — “Teacher X teaches Subject Y to Batch Z.” All downstream academic logic (timetable, attendance) cascades from this one relationship
TimetableSlot	References a BatchSubjectTeacher plus day / start–end time / classroom
AttendanceSession / AttendanceRecord	Per-class session header and per-student status records
StudyMaterial / Homework / HomeworkStatus	Content publishing and per-student completion tracking
Exam / Mark	Added in Session 9 — exam definitions and per-student scores, bounded by maxMarks
ParentStudentLink	Many-to-many link between parent and student User rows

3.2 Design Decisions

- **The golden rule:** every table carries instituteId, so an API route will fail loudly if a developer forgets to scope a query — there is no code path that silently leaks cross-tenant data.
- **Soft delete for people, hard delete for structure:** Teachers and Students are deactivated via isActive: false (they're referenced by historical attendance and marks); Batches, Subjects, Assignments, and Timetable slots are hard-deleted since they're purely structural.
- **Read/write scoping can diverge:** for Study Materials and Homework, read access follows a teacher's *current* BatchSubjectTeacher assignment, while write access follows original ownership (uploadedBy / assignedBy) — so a teacher moved to a new batch doesn't lose the ability to fix content they created.
- **Homework rosters are snapshots:** HomeworkStatus rows are created once, at assignment time, for every currently active student in the batch — not recomputed live.
- **Junction table over arrays:** BatchSubjectTeacher was chosen over ad-hoc many-to-many arrays specifically so Timetable and Attendance could attach cleanly to one well-defined relationship instead of three loosely coupled fields.

4. API Reference

Prabodha exposes **45+ REST endpoints** across 9 resource groups. Every endpoint except registration and login requires an `Authorization: Bearer <token>` header. The tables below are the complete, current surface.

4.1 Authentication — /api/auth/

Method	Endpoint	Access	Description
POST	/api/auth/register-institute	Public	Register a new institute + admin account
POST	/api/auth/login	Public	Log in with email + password, returns a JWT
POST	/api/auth/logout	Any auth	Invalidate session (client-side token clear)
GET	/api/auth/me	Any auth	Get the current user's profile
POST	/api/auth/create-user	Admin	Create a new user (teacher / student / parent)
POST	/api/auth/forgot-password	Public	Request a password-reset token
POST	/api/auth/reset-password	Public	Reset a password using a reset token

4.2 Batches & Subjects

Method	Endpoint	Access	Description
GET	/api/batches	Any auth	List all batches, with student counts
POST	/api/batches	Admin	Create a batch
GET	/api/batches/:id	Any auth	Batch detail with students + subject-teacher assignments
PATCH	/api/batches/:id	Admin	Update batch name
DELETE	/api/batches/:id	Admin	Hard delete batch
POST	/api/batches/:id/students	Admin	Bulk-assign students to a batch
GET	/api/subjects	Any auth	List all subjects
POST	/api/subjects	Admin	Create a subject
GET	/api/subjects/:id	Any auth	Subject detail
PATCH	/api/subjects/:id	Admin	Update subject name
DELETE	/api/subjects/:id	Admin	Hard delete subject

4.3 Teachers, Students & Parents

Method	Endpoint	Access	Description
GET	/api/teachers	Any auth	List active teachers (?includeInactive= to include deactivated)

Method	Endpoint	Access	Description
POST	/api/teachers	Admin	Create a teacher
GET	/api/teachers/:id	Any auth	Teacher detail
PATCH	/api/teachers/:id	Admin	Update teacher profile
DELETE	/api/teachers/:id	Admin	Soft delete (isActive: false)
GET	/api/students	Admin / Teacher	List students — admin: all; teacher: only assigned batches
POST	/api/students	Admin	Create a student
GET	/api/students/:id	Admin / Teacher	Student detail — 403 if teacher isn't assigned to that batch
PATCH	/api/students/:id	Admin	Update student profile
DELETE	/api/students/:id	Admin	Soft delete (isActive: false)
GET	/api/parents	Admin	List active parents with linked students
POST	/api/parents	Admin	Create a parent
GET	/api/parents/:id	Admin	Parent detail with linked students
PATCH	/api/parents/:id	Admin	Update parent profile
DELETE	/api/parents/:id	Admin	Soft delete (isActive: false)

4.4 Assignments & Timetable

Method	Endpoint	Access	Description
GET	/api/assignments	Admin / Teacher	List BatchSubjectTeacher rows — admin: all, filterable; teacher: own only
POST	/api/assignments	Admin	Create an assignment (validates batch/subject/teacher membership)
GET	/api/assignments/:id	Admin / Teacher	Assignment detail — teacher: own only, else 403
PATCH	/api/assignments/:id	Admin	Reassign the teacher on an assignment
DELETE	/api/assignments/:id	Admin	Hard delete assignment
GET	/api/timetable	Admin / Teacher	List slots — admin: all, filterable; teacher: own schedule only
POST	/api/timetable	Admin	Create a slot; checks teacher/batch/classroom conflicts
GET	/api/timetable/:id	Admin / Teacher	Slot detail
PATCH	/api/timetable/:id	Admin	Update day / time / classroom; re-checks conflicts
DELETE	/api/timetable/:id	Admin	Hard delete slot

4.5 Attendance, Materials, Homework & Marks

Method	Endpoint	Access	Description
GET	/api/attendance	Admin / Teacher	List sessions with a present/absent summary
POST	/api/attendance	Admin / Teacher	Create a session + all per-student records in one call
GET	/api/attendance/:id	Admin / Teacher	Full session detail with per-student records
PATCH	/api/attendance/:id	Admin / Teacher	Correct existing attendance records
DELETE	/api/attendance/:id	Admin / Teacher	Hard delete (cascades to records)
GET	/api/materials	Admin / Teacher / Student	List materials — student access scoped to own batch
POST	/api/materials	Admin / Teacher	Publish a material (link, PDF, note, or image)
PATCH · DELETE	/api/materials/:id	Owner / Admin	Edit or remove — write access follows uploadedBy
GET	/api/homework	Admin / Teacher / Student	List homework — student access scoped to own batch
POST	/api/homework	Admin / Teacher	Assign homework; auto-creates a HomeworkStatus snapshot per active student
PATCH	/api/homework/:id	Admin/Teacher · Student	Bulk status correction, or a student's single self-only status update
GET	/api/exams	Admin / Teacher	List exams with a computed studentsGraded / average summary
POST	/api/exams	Admin	Create an exam definition
PATCH	/api/exams/:id	Admin / Teacher	Upsert marks — create-or-correct per student

4.6 Parent–Student Links

Method	Endpoint	Access	Description
GET	/api/parent-student-links	Admin	List all links, filterable by parentId / studentId
POST	/api/parent-student-links	Admin	Create a link; validates roles and institute ownership; 409 on duplicate
DELETE	/api/parent-student-links/:id	Admin	Hard delete link

Not yet implemented *at the time earlier drafts of the roadmap were written — Attendance, Study Materials, Homework, Marks, and Institute Profile APIs — have since all been built and tested; the tables above reflect the current, completed surface.*

5. Security & Data Protection

Concern	Mitigation
SQL Injection	Prisma ORM parameterises every query — raw SQL string concatenation never occurs
Cross-Site Scripting (XSS)	React auto-escapes all JSX-rendered variables, neutralising script tags stored in the database
Cross-Site Request Forgery	Auth relies on a custom Authorization: Bearer header rather than cookies, so browsers never auto-attach credentials to forged cross-origin requests
Password storage	Passwords are bcrypt-hashed with a salt round before insertion; never stored in plain text
Cross-tenant data leakage	The apiHandler / requireAuth wrapper guarantees a route fails rather than silently returning unscoped data if instituteId scoping is omitted
Password strength on creation	Dual-layered validation — HTML minLength=8 + client trim() on the create form, and a Zod z.string().min(8) check server-side

5.1 A Concrete Bug Class This Design Caught

During Session 5, teacher-scoped queries across the Students and Assignments routes used `user.id` instead of the token payload's actual `user.sub` field. Because Prisma treats an `undefined` filter value as “no filter,” the bug failed *silently* for list endpoints — teacher-scoped student lists were quietly returning unscoped, institute-wide results — while failing loudly (a permanent 403) on direct equality checks. It was caught while testing the Assignments detail route, traced to four call sites, and fixed with regression re-tests across the Students teacher-scoping suite. This is recorded here because it is a useful, general lesson: an authorization filter that silently degrades to “show everything” is more dangerous than one that fails loudly, and is worth testing for explicitly.

6. Development Timeline

Prabodha has been built through a disciplined session-based workflow — every working session is required to produce an implementation plan, a daily task checklist, a walkthrough, and a changelog entry. The table below consolidates the project's changelog into a single chronological record of what was built, session by session.

Date	Focus	Outcome
Jul 23, AM	**Project cleanup & reorganisation**	Audited 46 root files; archived legacy Express/SQL code; root file count cut from 46 to 13
Jul 23, PM	**Status review & Git initialisation**	Full inventory of completed work; Git repo initialised and first commit (75 files) pushed to GitHub
Jul 23, Night	**Teachers API**	CRUD with admin-only writes and soft delete; established the apiHandler + Zod route convention
Jul 24	**Students API**	Teacher-scoped list/detail access derived from BatchSubjectTeacher, not a separate permissions table
Jul 24	**Assignments API**	BatchSubjectTeacher CRUD; replaced the manual Prisma Studio workaround used for earlier testing
Jul 24	**Bug fix — user.sub vs user.id**	Found and fixed a silent authorization-scoping bug across four route files (see §5.1)
Jul 24	**Timetable API**	TimetableSlot CRUD with application-level conflict detection for teacher / batch / classroom double-booking
Jul 24	**Attendance API**	AttendanceSession + AttendanceRecord CRUD; uniquely, both admin and the assigned teacher can write
Jul 24	**Study Materials + Homework APIs**	Introduced the read-by-current-assignment / write-by-ownership access split
Jul 24	**Marks/Exams schema + API, first refactor**	Added Exam and Mark models; extracted shared content-scope.ts helper
Jul 24	**Dashboard wiring — AdminResourcePage**	Batches, Subjects, Materials, Homework, and Assessments all switched from the InstitutionStore mock to live API calls
Jul 24	**Frontend auth foundation**	AuthProvider context, real login page, and a working route guard on the dashboard layout
Jul 24	**Parent-Student Linking + Directory wiring**	ParentStudentLink CRUD; DirectoryPage moved off the localStorage mock
Jul 24	**Attendance UI + documentation correction pass**	Real teacher-facing attendance UI; corrected a badly stale ROADMAP.md and created api-reference.md
Jul 24	**Bug fix — misnamed route file**	parent-links/[id]/rote.ts typo meant the single-link GET/DELETE route was never registered by Next.js
Jul 24	**Timetable UI**	Assignment-picker form replacing three free-text fields; fixed an infinite-fetch loop on the Attendance page
Jul 24	**Study Materials UI**	Drive-like grouped-by-subject view; extended student read access, scoped to their own batch
Jul 24	**Homework UI**	Teacher assign/grade view plus a stricter, self-only student status-update endpoint

Date	Focus	Outcome
Jul 25	**Login page, AuthProvider & auth guard**	Branded login route with institute-code memory; session hydration via GET /api/auth/me
Jul 26	**Password input & Parents navigation**	Editable initial-password field (default Welcome@123) with dual-layer validation; Parents nav + placeholder page
Jul 26	**Parent-Student Linking — full build**	Parents API, links API, and a complete ParentsPage management UI with a link/unlink modal

The changelog also records two other engineering-hygiene bugs beyond the two above: an `AttendanceSession` *route that assumed an undeclared Prisma relation (fixed by resolving* `BatchSubjectTeacher` *rows manually), and a reconstructed log entry for one session whose original notes were lost — flagged explicitly as a reconstruction rather than presented as a verified record.*

7. Master Roadmap & Progress

The roadmap tracks seven phases from foundation through deployment. Phases 1–5 — the entire backend, database, and role-specific dashboard UI — are complete. Phases 6 and 7 (production polish and deployment) remain open.

Phase 1 — Foundation & Scaffold ✓ COMPLETED

✓	Project cleanup, legacy code archival, Next.js + Tailwind scaffold
✓	Prisma schema (13 relational tables) and JWT auth middleware
✓	Dashboard UI skeleton; Git initialised and connected to GitHub
✓	SQLite local dev database; Batches and Subjects APIs (CRUD + bulk assignment)

Phase 2 — Core Entity APIs & UI Wiring ✓ COMPLETED

✓	Teachers and Students APIs with soft-delete semantics
✓	Parent-Student linking API
✓	DirectoryPage and AdminResourcePage rewired from the InstitutionStore mock to real APIs

Phase 3 — Academic Operations ✓ COMPLETED

✓	Timetable API and a visual scheduling grid for admins / read-only view for teachers & students
✓	Attendance API and a fast, teacher-facing in-class marking UI

Phase 4 — Content & Assessments ✓ COMPLETED

✓	Study Materials API + drive-like UI
✓	Homework API + teacher assign/grade and student self-service completion UI
✓	Marks/Grades schema, API, and UI

Phase 5 — Role-Specific Dashboards ✓ COMPLETED

✓	Teacher dashboard restricted to BatchSubjectTeacher-assigned students
✓	Student dashboard: read-only timetable, homework, and materials
✓	Parent dashboard: read-only summary of linked students' attendance, homework, and marks

Phase 6 — Polish & Production Readiness ● TODO

■	Toast-based error handling for graceful API-error display
■	Skeleton loaders for all data-fetching components
■	Migrate the Prisma provider from SQLite to PostgreSQL

■	Seed script for the first production institute registration

Phase 7 — Deployment ● TODO

■	VPS provisioning — Ubuntu, Node.js, PM2, Nginx
■	PostgreSQL hosting (self-managed or managed database)
■	Secure production environment variables and database URLs
■	Domain + Let's Encrypt SSL
■	Optional CI/CD via GitHub Actions on merge to main

8. Scalability Strategy

8.1 Database Scaling

- **Phase 1 — PostgreSQL:** before launch, the schema migrates from SQLite to PostgreSQL, which supports the concurrent writes needed for morning attendance rushes.
- **Phase 2 — read replicas:** roughly 90% of LMS traffic is read-heavy (students viewing timetables, parents viewing marks); Prisma read operations can be routed to replicas while writes stay on the primary node.

8.2 Application Scaling

- **Stateless architecture:** because sessions are JWT-based, the Next.js server holds no session state — 10 identical Node.js instances can sit behind an Nginx load balancer, and any instance can serve any request.
- **Caching headroom:** Redis can later cache timetable and syllabus data, reducing database hits to near zero for largely static information.

8.3 Concurrency Reasoning

Node.js's asynchronous, event-driven model means a standard VPS running Next.js can hold thousands of concurrent connections comfortably; the realistic bottleneck is the database, not the application server. Prisma's connection pooling is the intended mitigation for exactly the scenario that matters most operationally: hundreds of students marking in, or teachers submitting attendance, within the same few minutes each morning.

9. Future Roadmap

Once the V1 MVP is deployed, the architectural foundation supports rapid expansion in four directions:

#	Direction	Description
1	**Payment Gateway Integration**	Razorpay / Stripe integration to track student fee instalments automatically.
2	**Dedicated Mobile App**	A React Native app reusing the exact same REST APIs to power native parent communication.
3	**AI Analytics**	LLM-driven analysis of exam marks over time, auto-generating “Weak Subject Area” reports for parents.
4	**Online Examinations**	A locked-down MCQ testing portal that auto-grades directly into the Marks table.

Appendix — Mentor Q&A Notes

A short prepared reference for explaining Prabodha's design decisions in a review or interview setting.

Q: How does authentication work — session vs JWT?

JWTs were chosen over server-side sessions. On login the server issues a cryptographically signed token; because the server never has to remember session state in RAM or the database, the backend is fully stateless and horizontally scalable. The client attaches the token via the Authorization header on every request.

Q: Why Next.js instead of a separate React + Express backend?

Development speed and shared types. The App Router lets React Server Components and API Route Handlers live in the same folder structure, sharing TypeScript types end-to-end and removing the need for a separate API-sync layer.

Q: Why is the schema designed around BatchSubjectTeacher?

Almost all academic logic reduces to one fact: “Teacher X teaches Subject Y to Batch Z.” Making that fact a dedicated junction table means Timetable slots and Attendance sessions can attach cleanly to a single, well-defined relationship instead of three loosely coupled fields.

Q: What happens if 1,000 students log in at once?

Node's async, event-driven model handles thousands of concurrent connections without difficulty; the real bottleneck is the database, mitigated with Prisma connection pooling so an attendance rush doesn't overwhelm it.